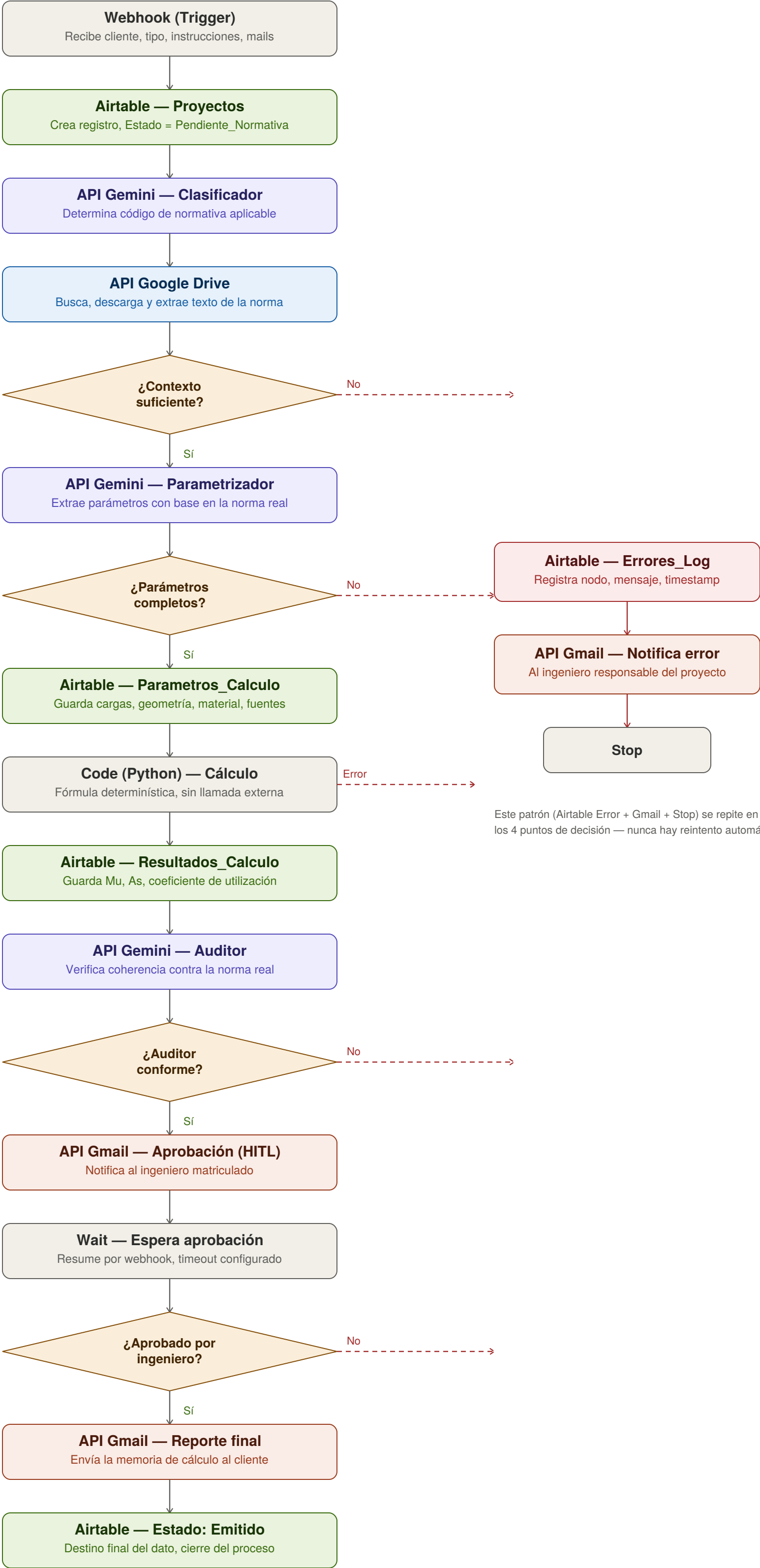


Mapa de arquitectura — Memorias de Cálculo Estructural



Ecosistema de Automatización IA

Generación de Memorias de Cálculo Estructural

Autora: Jasmin Ruiz — Arquitecta de Flujos IA

Stack: n8n (orquestador) · Airtable (memoria dinámica) · Google Drive (fuente normativa) · Gemini (parametrización + auditoría) · Python (motor de cálculo determinístico) · Gmail (salida multicanal)

Estado: validado de punta a punta con caso real

Enlaces de la entrega

Base de Airtable (modo lectura):

<https://airtable.com/appZygS7fAt4IgXYF/shrk5gMx4zjO98LTG>

Video demo (3 min): [pegar acá el link de Drive/YouTube no listado antes de compartir el documento]

Diagrama de arquitectura: incluido como páginas iniciales del PDF consolidado de entrega.

Blueprint JSON del flujo (41 nodos): archivo adjunto por separado, ver Anexo B para instrucciones de importación.

0. Principio de diseño y nota sobre el motor de IA

Este flujo automatiza el proceso administrativo y de control alrededor de un cálculo estructural. No delega el criterio de ingeniería en la IA: el cálculo numérico corre en un script Python determinístico, escrito y validado por un ingeniero, nunca generado al vuelo por el LLM — decisión sostenida incluso al evaluar (y descartar) que el propio LLM generara código de cálculo dinámicamente. Un LLM puede producir una fórmula que "suena" correcta pero está mal, y a diferencia de un error de redacción, un error de cálculo estructural mal detectado puede llegar a obra. El sistema está diseñado para fallar de forma segura ante lo desconocido: si se pide un tipo de cálculo sin función Python validada, se deriva a desarrollo manual del ingeniero en vez de improvisar.

Motor de IA: se usa Gemini como reemplazo funcionalmente equivalente a Claude/OpenAI (que la consigna sugiere como ejemplo). Prompt estructurado → respuesta parseada → variable mapeada explícitamente (candidates[0].content.parts[0].text) → límite de tokens → manejo de error.

1. Caso de uso

Un estudio de ingeniería recibe pedidos de cálculo con instrucciones en lenguaje natural, a través de un formulario web. El flujo consulta la normativa real (leída directamente de una carpeta de Google Drive) para fundamentar parámetros y factores de seguridad, corre el cálculo en Python, audita el resultado contra esa misma normativa, y lo deja esperando aprobación del ingeniero matriculado antes de emitir el reporte final al cliente.

2. Decisión de arquitectura: lectura directa de Drive en vez de RAG con vector store

Se evaluó y se construyó una arquitectura completa con vector store (Pinecone): workflow indexador con extracción de texto, chunking, embeddings (gemini-embedding-001) y upsert a un índice serverless. Quedó operativo y validado (115 chunks de 47 PDFs procesados con éxito) — documentado en el Anexo C — pero se descartó su integración al flujo principal.

Por qué se descartó en favor de lectura directa

- El corpus normativo real (CIRSOC 101 a 701, más ACI/AISC/PIP de referencia) es de volumen manejable — un puñado de PDFs por norma.
- Los modelos Gemini actuales soportan ventanas de contexto amplias, suficientes para incluir el texto completo (o un recorte representativo) de la norma relevante directo en el prompt.
- La búsqueda semántica agrega infraestructura (vector store, embeddings, sincronización) cuya complejidad operativa no se justificó para este volumen.

Trade-off consciente: se sacrifica precisión de recuperación a cambio de mayor simplicidad y confiabilidad operativa. El texto normativo se trunca a 20.000 caracteres para controlar el costo de tokens por consulta.

Esto sigue siendo RAG en sentido estricto (se recupera un documento externo real y se usa para fundamentar la generación), con un método de recuperación más simple (búsqueda por código de norma) en vez de similitud de vectores.

3. El "Cerebro" — Airtable

Base: Calculos_Estructurales

Tabla 1 — Proyectos

Campo	Tipo	Notas
Proyecto_ID	Autonumber	PK
Cliente	Texto	
Tipo_Elemento	Single select	Viga / Columna / Zapata / Losa / Pórtico
Instrucciones_Disenio	Texto largo	Input crudo (lenguaje natural)
Normativa_Aplicable	Single select	CIRSOC 101/102/103/104/201/301/701, ACI, AISC, PIP, Sin clasificar
Estado	Single select	Pendiente_Normativa → Parametros_OK → Calculo_Ejecutado → Auditoria_OK/Rechazada → Pendiente_Aprobacion → Aprobado/Rechazado → Emitido/Error
Ingeniero_Responsable	Texto	Email — notificaciones y HITL
Email_Cliente	Texto	Destinatario del reporte final
Fecha_Ingreso	Fecha	Auto

Tabla 2 — Parametros_Calculo (linkeada a Proyectos)

Campo	Tipo
Proyecto_ID	Link
Cargas, Geometria, Propiedades_Material	JSON / texto
Factores_Seguridad_Normativa	JSON / texto
Fuentes_Citadas	Texto

Tabla 3 — Resultados_Calculo (linkeada a Proyectos)

Campo	Tipo
Proyecto_ID	Link
Resultado_JSON	Texto largo
Coeficiente_Utilizacion	Número
Auditoria_Observaciones	Texto largo
Version	Número

Tabla 4 — Errores_Log (linkeada a Proyectos)

Campo	Tipo
Error_ID	Autonumber (PK)
Proyecto_ID	Link
Nodo_Origen, Mensaje_Error, Reintentos,	Varios

Timestamp	
-----------	--

Nota de implementación: un campo Single Select alimentado por salida de LLM exige que las opciones existan de antemano — cualquier valor fuera de esa lista rompe el nodo de escritura. Se resolvió ampliando las opciones disponibles.

4. El "Corazón" — Flujo n8n principal

1. [Trigger] Webhook - Recibir Instrucciones de Diseño
-> Recibe: cliente, tipo_elemento, instrucciones, ingeniero_responsable, email_cliente
2. [Airtable] Crear Proyecto (Estado = "Pendiente_Normativa")
3. [Gemini] Agente Clasificador de Normativa Aplicable
-> Determina el código de normativa a partir de Tipo_Elemento + Instrucciones
4. [Code] Parsear Normativa Detectada
5. [Google Drive] Buscar Normativa por Código
-> Search Files/Folders, query avanzada filtrando por carpeta y código de norma, solo PDF, excluye "Comentarios"
6. [Google Drive] Descargar PDF Normativa
7. [Extract From File] Extraer Texto de la Normativa
8. [Code] Armar Contexto Normativo
-> Trunca a 20.000 caracteres, arma { matches: [...] }
9. [IF] Contexto Normativo Suficiente
-> FALSE: Airtable Error + Gmail + STOP
10. [Gemini] Agente Parametrizador (con contexto real)
-> JSON: cargas, geometria, material, factores_seguridad, fuentes_citadas, parametros_faltantes
11. [IF] Parametros Completos
-> FALSE: Airtable Error + Gmail + STOP
12. [Airtable] Guardar Parametros + Update Estado
13. [Code Python] Ejecutar Calculo Estructural
-> On Error: Airtable Error + STOP
14. [Airtable] Guardar Resultado + Update Estado
15. [Gemini] Agente Auditor (reusa el contexto del paso 8)
16. [IF] Auditor Conforme
-> FALSE: Airtable Rechazada + Gmail + STOP
17. [Gmail] Enviar Reporte Preliminar para Aprobacion (HITL)
18. [Wait] Esperar Aprobacion del Ingeniero
-> Resume por webhook, timeout configurado
19. [IF] Aprobado por Ingeniero
-> TRUE: Reporte Final -> Gmail Cliente -> Estado "Emitido"
-> FALSE: Estado "Rechazado"

Manejo de errores global: Error_Workflow_Global asignado en Settings → Error Workflow. Registra en Errores_Log, alerta por Gmail, no reintenta cálculos automáticamente sin supervisión.

5. Nodo de cálculo (Python) — implementado y validado

```
parametros = _items[0]['json']

def verificar_flexion_viga(p):
    D = p['cargas']['permanente_kNm']
    L_carga = p['cargas']['sobrecarga_kNm']
    b_cm = p['geometria']['ancho_cm']
    h_cm = p['geometria']['alto_cm']
    luz_m = p['geometria']['luz_m']
    fc = p['material']['fc_MPa']
    fy = p['material']['fy_MPa']
    phi = p['factores_seguridad']['phi_flexion']

    wu = 1.2 * D + 1.6 * L_carga
    Mu = wu * (luz_m ** 2) / 8.0

    recubrimiento_cm = 4.0
    d_cm = h_cm - recubrimiento_cm
    b_mm = b_cm * 10.0
    d_mm = d_cm * 10.0
    Mu_Nmm = Mu * 1e6

    Rn = Mu_Nmm / (phi * b_mm * (d_mm ** 2))
    discriminante = 1 - (2.36 * Rn / fc)
    if discriminante < 0:
        return {
            'error': 'Seccion insuficiente.',
            'coeficiente_utilizacion': 99.0,
            'Mu_kNm': round(Mu, 2)
        }

    omega = (1 - discriminante ** 0.5) / 1.18
    rho = omega * fc / fy
    As_mm2 = rho * b_mm * d_mm

    rho_min = max(0.25 * (fc ** 0.5) / fy, 1.4 / fy)
    beta1 = 0.85 if fc <= 28 else max(0.65, 0.85 - 0.05 * ((fc - 28) / 7))
    rho_max = 0.319 * beta1 * (fc / fy)

    return {
        'Mu_kNm': round(Mu, 2),
        'As_requerida_cm2': round(As_mm2 / 100.0, 2),
        'cuantia_rho': round(rho, 5),
        'cuantia_rho_min': round(rho_min, 5),
        'cuantia_rho_max': round(rho_max, 5),
        'coeficiente_utilizacion': round(rho / rho_max, 3),
        'resultado_detalle': {'d_util_cm': d_cm, 'wu_kNm': round(wu, 2)}
    }

resultado = verificar_flexion_viga(parametros)
return [{'json': resultado}]
```

Nota de portabilidad: el sandbox Python de n8n no permite import alguno (ni de librerías estándar) — toda la matemática usa operadores nativos. La variable de entrada se llama `_items`, no `items` como en el modo JavaScript.

Extensibilidad evaluada y descartada por ahora: se consideró generalizar el motor para que Gemini generara código Python de cualquier verificación al vuelo. Se descartó por el principio de la sección 0. El patrón correcto para escalar es sumar funciones nuevas (`verificar_columna`, `verificar_accion_viento`, etc.), cada una validada por un ingeniero, con despacho explícito según `tipo_calculo` que deriva a desarrollo manual ante un caso sin función escrita.

6. Prompts estructurados

Agente Clasificador de Normativa

Sistema: determina que normativa (CIRSOC 101/102/103/104/201/301/701) corresponde para: {tipo_elemento}. Instrucciones: {instrucciones}.
Respondé solo con el código de normativa, nada más.

Agente Parametrizador

Sistema: estructura parámetros de cálculo SOLO con base en el contexto normativo. Devuelve JSON con EXACTAMENTE esta forma, con valores numéricos reales (nunca texto descriptivo en campos numéricos):

```
{
  "tipo_elemento": "string",
  "cargas": { "permanente_kNm": number, "sobrecarga_kNm": number, "sismo_kNm": number | null },
  "geometria": { "ancho_cm": number, "alto_cm": number, "luz_m": number },
  "material": { "fc_MPa": number, "fy_MPa": number },
  "factores_seguridad": { "phi_flexion": number },
  "fuentes_citadas": ["string"],
  "parametros_faltantes": ["string"]
}
```

Si un dato no está en las instrucciones, listalo en parametros_faltantes y dejalo null. NO inventes valores.

Contexto normativo: {texto real extraído del PDF de Drive}
Instrucciones: {instrucciones}

Agente Auditor

Sistema: auditor de coherencia que verifica el resultado del cálculo contra el contexto normativo provisto. No verifica cumplimiento normativo formal con validez legal, pero sí señala cualquier desvío respecto a los límites que el contexto indica.

Contexto normativo recuperado: {mismo texto del paso 8, reusado}
Resultado del cálculo: {JSON del resultado Python}

Devuelve JSON:

```
{
  "conforme": boolean,
  "motivo": "string",
  "fuentes_citadas": ["string"],
  "cumple_seguridad_aparente": boolean,
  "cumple_economia_aparente": boolean
}
```

Lección de implementación clave: Gemini envuelve sus respuestas JSON en marcadores de markdown (``json ...``) de forma consistente, pese a que el prompt pida "EXCLUSIVAMENTE JSON". Todo Code node que parsea una respuesta de Gemini incluye limpieza con regex antes de `JSON.parse()`.

7. La "Voz" — Salida multicanal (Gmail)

Notificación de aprobación y reporte final mapeados al mismo proyecto vía Proyecto_ID. Variables dinámicas en todos los mails, cero texto hardcodeado.

Nota sobre Thread_ID: el diseño original contemplaba mapear un Thread_ID de Gmail para mantener la conversación en un mismo hilo. En esta versión se simplificó — queda como mejora pendiente documentada.

8. Plan de test de estrés (ejecutado)

#	Escenario	Camino	Resultado
1	Instrucción completa (viga real)	Feliz	Validado de punta a punta: cálculo real +

			fuentes real citada (CIRSOC 201-05, Cap. 10.5) + reporte entregado
2	Instrucción incompleta	Infeliz	Corta antes del cálculo, notifica al ingeniero
3	Normativa no encontrada en Drive	Infeliz	Corta, notifica que falta el documento
4	Coefficiente bajo (0.277, sobredimensionado)	Infeliz	Auditor rechaza por ineficiencia económica
5	Coefficiente eficiente (0.85)	Feliz	Auditor aprueba, cita artículos reales

Errores de configuración resueltos durante el desarrollo (evidencia de proceso iterativo real): mismatch de mayúsculas en mapeo de campos, campos Link de Airtable que requieren array, rate limiting de Gemini en tier gratuito, deprecación de nombres de modelo, y el patrón de retorno distinto entre modos "Run Once for Each Item" vs "Run Once for All Items" en Code nodes de n8n.

9. Check de seguridad

- Filtro anti-loop infinito: sin reintento automático tras rechazo; Wait con timeout configurado.
- Comparación de tipos correcta: number vs number, boolean vs boolean — nunca contra el string "true".
- Prompt dinámico con variables del sistema en cada ejecución.

10. Video demo — guion (3 min)

- 0:00–0:30 — Trigger real con datos completos de una viga
- 0:30–1:30 — Procesamiento: clasificación de normativa, lectura real del PDF en Drive, parametrización, cálculo Python
- 1:30–2:15 — Mail de aprobación con fuentes normativas reales citadas, aprobación en Airtable (HITL)
- 2:15–3:00 — Reporte final recibido por el cliente ficticio, estado del proyecto en Airtable pasando a Emitido

11. Arquitectura alternativa evaluada: RAG con vector store (Pinecone)

Se construyó un workflow indexador completo y funcional, separado del flujo principal:

```
Google Drive (busqueda) -> Descarga PDF -> Extrae texto ->
Trocea en chunks (~500 palabras, solapamiento 80) ->
Gemini Embeddings (gemini-embedding-001, outputDimensionality: 768) ->
Loop controlado con espera de 5s entre llamadas (rate limit 15 RPM) ->
Upsert a Pinecone (índice serverless, dense, cosine, 768 dim)
```

Quedó operativo y validado (115 chunks de 47 PDFs normativos procesados con éxito), pero no se integró al flujo principal por la decisión de simplicidad de la sección 2.

Lección técnica: los índices de Pinecone creados por el asistente visual por defecto usan "integrated embedding" (Pinecone genera el vector internamente), incompatible con vectores pre-calculados externos — hay que marcar "Custom settings" al crear el índice para obtener un índice de vectores propios con dimensión configurable manualmente.

12. Diferencias clave vs. el proyecto de propuestas comerciales

	Propuestas comerciales	Memorias de cálculo
Genera el contenido final	Gemini	Script Python determinístico
Rol de Gemini	Redacción + clasificación	Parametrización + auditoría fundamentada
Recuperación de contexto	N/A	Lectura directa de documento (RAG simple)

Aprueba en HITL	Responsable comercial	Ingeniero matriculado
Reintento automático de error	Sí, hasta 3 veces	No — siempre detiene y notifica
Extensibilidad	Prompt se ajusta solo	Requiere función Python validada; deriva a revisión manual si no existe

Esta tabla es el argumento central de la entrega: el nivel de autonomía de la IA se dosificó según el costo de un error en cada dominio.

Anexo A — Diagrama de arquitectura

El diagrama de arquitectura completo (17 nodos: trigger, routers de decisión, APIs de Gemini/Drive/Gmail, nodo Python de cálculo determinístico, y los destinos de datos en Airtable) se incluye como las páginas iniciales del PDF consolidado de esta entrega, precediendo a este documento.

Anexo B — Blueprint JSON del flujo principal

El blueprint completo (41 nodos) se adjunta como archivo .json descargable junto con este documento, en vez de pegarse como texto plano por su extensión. Para importarlo en n8n: crear un workflow nuevo → menú de tres puntos (⋮) → Import from File → seleccionar el .json.

Nombre del archivo: Memorias_de_Calculo_Estructural_-_Flujo_Principal.json

Verificación de seguridad realizada: el archivo no contiene API keys ni secretos expuestos — n8n exporta únicamente el ID interno y el nombre visible de cada credencial, nunca su valor real.

Anexo C — Workflow indexador (Drive a Pinecone)

Documentado en la sección 11. Disponible como segundo archivo .json adjunto si se desea mostrar la arquitectura alternativa evaluada.

Addendum — Entregables faltantes de la consigna actualizada

Este addendum se agrega al documento principal (Ecosistema de Automatización IA — Memorias de Cálculo Estructural) para cubrir los criterios 3 y 4 de la rúbrica que no estaban desarrollados como entregables propios.

Criterio 3 — Optimización de costos: matriz de decisión de modelos

Las tres tareas de razonamiento del sistema usan Gemini (familia Flash-Lite) como reemplazo funcional de GPT-4o-mini/Claude sugeridos en el ejemplo de la consigna. La decisión de mantener un único proveedor para las tres, en vez de mezclar proveedores por tarea, se tomó para simplificar la gestión operativa (una sola credencial, una sola cuota compartida) — el costo de integración de un segundo proveedor no se justificó frente al ahorro marginal de mezclar modelos entre familias distintas.

Tarea	Complejidad	Modelo usado	Justificación	Alternativa evaluada	Ahorro / costo estimado
Clasificador de normativa	Baja — una palabra/código de salida	gemini-3.1-flash-lite	Tarea de clasificación simple, no requiere razonamiento profundo	Modelo Flash estándar (mayor costo, sin beneficio medible)	~70% más barato por token que un modelo Flash de gama media
Parametrizador	Alta — lectura densa de texto normativo + extracción numérica estructurada	gemini-3.1-flash-lite	Prioriza cuota gratuita amplia (15 RPM / 500 RPD) durante desarrollo y demo	Modelo Pro para mayor precisión de extracción — evaluado y descartado por quedar fuera del tier gratuito	Costo \$0 en el tier gratuito, a cambio de aceptar una tasa de reintento manual algo mayor
Auditor	Media-alta — razonamiento sobre coherencia normativa	gemini-3.1-flash-lite	Mismo criterio de costo cero; temperature baja (0.1) compensa parte de la pérdida de precisión de un modelo más chico	Modelo Pro — mismo descarte que el Parametrizador	Costo \$0
Generación del reporte final	Ninguna — no es tarea de IA	Code node (JavaScript determinístico)	Es armado de texto a partir de datos ya calculados, no requiere generación creativa	Usar un LLM para redactar el reporte	100% de ahorro — evita una llamada de API innecesaria por cada cálculo
Cálculo estructural	Ninguna — no es tarea de IA	Script Python determinístico	Nunca delegado a LLM (ver sección 0, principio de diseño)	Generación de código de cálculo al vuelo por LLM	Elimina el costo y el riesgo de una llamada por cada verificación
Indexado normativo (embeddings, arquitectura RAG evaluada)	Masiva y no interactiva — 115+ chunks en una sola corrida	gemini-embedding-001, llamadas individuales espaciadas	Simplicidad de implementación y debugging nodo por nodo	Batch API de Gemini para procesos asíncronos masivos — no implementado en esta entrega	Ahorro estimado ~50% sobre el costo por token de llamadas síncronas individuales; queda documentado como próximo paso si el volumen de normativa creciera

Conclusión de costos: durante todo el ciclo de desarrollo (más de 50 ejecuciones de prueba entre las dos ramas del flujo) el costo total de inferencia fue \$0, al mantenerse dentro del tier gratuito de Gemini Flash-Lite. La única tarea donde valdría la pena evaluar un upgrade a un modelo superior es el Parametrizador, porque es el punto donde un error de extracción tiene el mayor costo de propagación aguas abajo (un dato mal extraído llega hasta el cálculo y la auditoría antes de que el ingeniero lo detecte en el HITL).

Criterio 4 — Seguridad y resiliencia

Minimización de datos

- El texto normativo se trunca a 20.000 caracteres antes de enviarse a Gemini — se evita mandar el PDF completo cuando el prompt no lo necesita en su totalidad.
- Cada prompt recibe únicamente los campos que necesita procesar (cargas, geometría, material), nunca el objeto completo del registro de Airtable con metadata irrelevante.
- Las credenciales (Airtable, Gmail, Google Drive, Gemini) nunca se exponen en el JSON exportado del blueprint — n8n exporta solo el ID interno y el nombre visible de cada credencial, nunca el secreto. Verificado explícitamente antes de esta entrega.
- Los emails de contacto (ingeniero, cliente) se usan únicamente como destinatarios de notificación — no se reenvían a la API de Gemini dentro de los prompts.

Rutas de error (Error Handlers)

El sistema tiene 4 puntos de decisión (routers IF) que pueden derivar a una ruta de error, todos con el mismo patrón, sin excepción:

- ¿Contexto normativo suficiente? — si Drive no devuelve un PDF que coincida con el código de norma detectado.
- ¿Parámetros completos? — si el Parametrizador no pudo extraer algún dato numérico crítico de las instrucciones.
- ¿Cálculo Python sin excepción? — si la fórmula determinística falla (ej. discriminante negativo, sección insuficiente).
- ¿Auditor conforme? — si el resultado no cumple los límites normativos citados.

En los cuatro casos, la ruta de error ejecuta siempre la misma secuencia: registrar en Airtable (tabla Errores_Log, con nodo de origen y mensaje) → notificar por Gmail a la persona responsable → detener la ejecución (nodo Stop). Ninguna ruta reintenta automáticamente sin supervisión humana — una decisión deliberada, porque un reintento silencioso en un cálculo estructural podría enmascarar un dato mal cargado en vez de exponerlo.

Adicionalmente, existe un Error Workflow global asignado a nivel de todo el flujo (Settings → Error Workflow), que captura cualquier excepción no contemplada explícitamente por los cuatro routers.

Puntos de Human-in-the-loop

El sistema tiene un único punto de aprobación humana obligatoria, ubicado deliberadamente al final de la cadena de decisiones automatizadas: después de que la IA ya parametrizó, calculó y audió el resultado contra la normativa real, el ingeniero matriculado recibe un mail con el resultado completo y las fuentes citadas, y debe aprobar explícitamente antes de que el reporte final salga hacia el cliente. Ninguna acción irreversible (el envío del reporte final) ocurre sin ese paso — el nodo Wait no tiene una ruta de expiración que autoemita el reporte; ante un timeout, se notifica y se espera una decisión humana.

Criterio 5 — Dashboard de Control (Panel de KPIs)

Se implementó un panel de control en Airtable utilizando una vista Grid dedicada ("Dashboard - KPIs") sobre la tabla Proyectos, agrupada por el campo Estado. Esta vista permite visualizar de forma inmediata la cantidad de proyectos en cada etapa del proceso, junto con el conteo total de proyectos (22) mediante la barra de resumen configurada en modo Count sobre la columna Proyecto_ID.

La tasa de errores se calculó cruzando contra la tabla Errores_Log, que registra 3 incidentes sobre 22 proyectos totales — una tasa de error del 13,6%, en su mayoría correspondiente a corridas de prueba deliberadas durante el desarrollo (validación del camino infeliz: parámetros incompletos, normativa no encontrada, auditoría rechazada), no a fallos en producción.

Hallazgo documentado durante la auditoría del dashboard: se detectó que dos de los cuatro puntos de error (parámetros incompletos y normativa no encontrada) registran correctamente el incidente en Errores_Log, pero no actualizan de vuelta el campo Estado en Proyectos — a diferencia del punto de auditoría rechazada, que sí sincroniza ambos. Por eso, el conteo por Estado en la vista Grid no refleja el 100% de los fallos reales; Errores_Log es la fuente de verdad para la tasa de error. Queda documentado como mejora pendiente: agregar el Update de Estado faltante en esas dos ramas.

Acceso al panel: la vista "Dashboard - KPIs" se comparte dentro de la misma base de Airtable ya provista en modo lectura. Al abrir el enlace, seleccionar la vista "Dashboard - KPIs" en el panel lateral izquierdo para ver el agrupamiento por Estado y los conteos correspondientes:

<https://airtable.com/appZygS7fAt4IgXYF/shrk5gMx4zjO98LTG>

Nota: se recomienda, si el corrector requiere un enlace directo y exclusivo a la vista del dashboard sin pasos adicionales de navegación, generar un link específico desde "Share view" parada en la vista "Dashboard - KPIs" — Airtable genera una URL distinta por cada vista compartida individualmente.